

Comenzado el	viernes, 29 de noviembre de 2024, 12:03
Estado	Finalizado
Finalizado en	viernes, 29 de noviembre de 2024, 12:17
Tiempo empleado	14 minutos 15 segundos
Puntos	4,50/9,00
Calificación	5,00 de 10,00 (50%)

Pregunta 1

Correcta

Se puntúa 1,00 sobre 1,00

El problema de la multiplicación encadenada de n matrices de forma óptima puede resolverse con una complejidad en tiempo en:

Seleccione una:

- ☐ a. $O(m^3)$ siendo m el número de filas y columnas de las n matrices
- ☐ b. $O(m^2)$ siendo m el número de filas y columnas de las n matrices
- ☒ c. $O(n^3)$ ✓
- ☐ d. $O(n^2)$

El algoritmo calcula para cada pareja de índices i, j tales que $i \leq j$ el número mínimo de multiplicaciones básicas para multiplicar las matrices de la i a la j . El cálculo de cada valor es lineal en n porque requiere considerar las distintas formas de colocar los paréntesis externos para realizar la última multiplicación. Puesto que la cantidad de parejas a calcular es del $O(n^2)$, el coste está en $O(n^3)$.

La respuesta correcta es: $O(n^3)$

Pregunta 2

Sin contestar

Sin calificar

Si modificamos el problema de justificar las palabras a ambos márgenes de un párrafo para que sea obligatorio completar las líneas y se penalice el número de letras que sobrepasan el margen, en lugar del espacio sobrante, existe un algoritmo voraz que resuelve el problema.

Seleccione una:

- ☐ a. Verdadero
- ☐ b. Falso

Verdadero. Con la modificación descrita no hay más remedio que saltar de línea en la primera palabra que complete la línea actual, tal vez excediéndose en el margen, y por tanto hay un algoritmo voraz para resolver el problema.

La respuesta correcta es: Verdadero

Pregunta 3

Sin contestar

Se puntúa como 0 sobre 1,00

¿Cuántas formas distintas hay de multiplicar secuencialmente n matrices de dimensiones compatibles?

Seleccione una:

- ☐ a. Depende de las dimensiones de cada una de las matrices.
- ☐ b. n^3
- ☐ c. $(n - 1)!$
- ☐ d. Depende de los valores de cada una de las matrices.

Si llamamos m_n a las formas distintas de multiplicar n matrices, $m_2 = 1$ porque solo hay una forma de multiplicar dos matrices, independientemente de sus dimensiones (son compatibles) y de sus valores. Si $n \geq 3$, tendremos que multiplicar un par contiguo de esas matrices de $n - 1$ posibles a elegir y eso deja una secuencia de $n - 1$ matrices que tendremos que seguir multiplicando, es decir, $m_n = (n - 1) m_{n-1}$. Luego $m_n = (n - 1)!$.

La respuesta correcta es: $(n - 1)!$

Pregunta 4

Sin contestar

Se puntúa como 0 sobre 1,00

En el problema del cambio de monedas con un sistema monetario de valores {1, 3, 5}, ¿de cuántas formas posibles se puede pagar la cantidad 10 con el menor número de monedas?

Seleccione una:

- ☐ a. 4
- ☐ b. 1
- ☐ c. 3
- ☐ d. 0

El mínimo número de monedas para pagar dicha cantidad es 2.

A medida que construimos la tabla de programación dinámica ascendente, aquí mostrada

	0	1	2	3	4	5	6	7	8	9	10
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	1	2	3	4	5	6	7	8	9	10
2	0	1	2	1	2	3	2	3	4	3	4
3	0	1	2	1	2	1	2	3	2	3	2

podemos rellenar una matriz que en cada posición (i, j) guarda el número de formas de obtener la cantidad j con el menor número de monedas usando los tipos 1 a i .

La columna 0 la rellenamos con unos y el resto de la fila 0 con ceros.

Para el resto de casos si $monedas[i-1][j]$ coincide con $monedas[i-1][j - M[i-1]] + 1$, entonces el número de formas para pagar j (usando el menor número de monedas) con las monedas de tipos 1 a i es el número de formas para pagar j sin usar la moneda i más el número de formas para pagar $j - M[i-1]$ usando los tipos 1 a i de nuevo. En este caso obtenemos la siguiente tabla:

	0	1	2	3	4	5	6	7	8	9	10
0	1	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	1	1	1	1
2	1	1	1	1	1	1	1	1	1	1	1
3	1	1	1	1	1	1	2	2	1	2	1

Y por tanto la solución es: 1

La respuesta correcta es: 1

Pregunta 5

Sin contestar

Se puntúa como 0 sobre 1,00

Supongamos un sistema monetario con cantidad ilimitada de monedas de valores {2, 5, 10}. Se desea pagar la cantidad 10 con el menor número de monedas. Escribe separados por un espacio y ordenados de menor a mayor los *índices de la última fila* de la matriz rellenada por el algoritmo *que contienen un infinito*.

Respuesta:



Las filas se rellenan de arriba abajo y de izquierda a derecha obteniendo la siguiente tabla:

	0	1	2	3	4	5	6	7	8	9	10
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	1	∞	2	∞	3	∞	4	∞	5
2	0	∞	1	∞	2	1	3	2	4	3	2
3	0	∞	1	∞	2	1	3	2	4	3	1

Por tanto la solución es: 1 3

La respuesta correcta es: 1 3

Pregunta 6

Correcta

Se puntúa 1,00 sobre 1,00

En el problema de la justificación de un texto, si queremos reconstruir la solución (decidir qué palabras van en cada línea) es necesario utilizar un espacio adicional en $O(nL)$, siendo n el número de palabras del texto y L la longitud de las líneas.

Seleccione una:

- ☐ a. Verdadero
- ☒ b. Falso ✓

Falso. Si para cada i calculamos la penalización mínima para formatear las palabras desde la i -ésima hasta la última comenzando con una línea en blanco basta con un espacio adicional en $O(n)$ siendo n el número de palabras del texto.

La respuesta correcta es: Falso

Pregunta 7

Incorrecta

Se puntúa -0,50 sobre 1,00

Al calcular el número combinatorio $\binom{n}{r}$ utilizando recursión sin memoria, el número total de llamadas recursivas realizadas está en $\theta(n^2)$.

Seleccione una:

- ☒ a. Verdadero ✖
- ☐ b. Falso

Falso. El número de veces que se repiten los términos $\binom{i}{j}$ que hacen falta para calcular $\binom{n}{r}$ está representado por un número combinatorio. Por ejemplo, el término $\binom{2}{1}$ se calcula $\binom{n-2}{r-1}$ veces. Se puede demostrar que $\binom{n}{n/2} \in \theta(2^n)$.

La respuesta correcta es: Falso

Pregunta 8

Correcta

Se puntúa 1,00 sobre 1,00

Se suele utilizar programación dinámica en lugar de recursión sin memoria cuando hay una gran cantidad de subproblemas repetidos.

Seleccione una:

- ☒ a. Verdadero ✔
- ☐ b. Falso

Verdadero. En algunos algoritmos recursivos se generan muchas llamadas repetidas, lo que redundaría en un coste elevado. La programación dinámica utiliza una tabla para almacenar los valores de los subproblemas ya calculados, de forma que en caso de requerirse varias veces no sea necesario volver a calcularlos sino solamente recuperarlos de la tabla.

La respuesta correcta es: Verdadero

Pregunta 9

Correcta

Se puntúa 1,00 sobre 1,00

La solución por programación dinámica al problema de la multiplicación encadena de matrices puede implementarse utilizando el método recursivo descendente.

Seleccione una:

- ☒ a. Verdadero ✔
- ☐ b. Falso

Verdadero. La programación dinámica utiliza una tabla para almacenar los valores de subproblemas ya calculados evitando repetir dichos cálculos. El rellenado de esa tabla puede realizarse de manera recursiva descendente y en tal caso solo se almacenarán aquellos valores de subproblemas que sean necesarios para calcular el resultado del problema original.

La respuesta correcta es: Verdadero

Pregunta 10

Correcta

Se puntúa 1,00 sobre 1,00

Supongamos una función recursiva que está definida de la siguiente manera:

$$f(1) = e_1$$

$$f(i) = \max(e_i, f(i-1) + e_i) \text{ para } i > 1$$

siendo $e_1, \dots, e_n$ expresiones numéricas y que queremos implementar un algoritmo de programación dinámica ascendente que calcule $\max_{i=1}^n f(i)$. ¿Qué cantidad de memoria adicional óptima necesita el algoritmo?

Seleccione una:

- ☐ a. $\theta(n^2)$
- ☒ b. $\theta(1)$ ✓
- ☐ c. $\theta(n \log n)$
- ☐ d. $\theta(n)$

Basta con utilizar una variable para guardar el valor de $f(i)$ que vamos calculando (desde 1 hasta n por ser ascendente), y otra para guardar el máximo de los valores de f calculados hasta el momento. Luego la respuesta correcta es $\theta(1)$.

La respuesta correcta es: $\theta(1)$